

Sri Lanka Institute of Information Technology

Enterprise Publishing Management System

FINAL REPORT

IE3151 - Information Systems Project Management

Project ID: ISE_WE_0101_22

Submitted by:

1. IT24101357– Bandara N.H.M.W.W.D.S
2. IT24101035– Sewwandi R.P.D.D.C
3. IT2410083– Ananda U.V.D.M.A
4. IT24100696– Dissanayake L.N

CONTENTS

1. INTRODUCTION 4

1.1 Project Background 4

1.2 Business Context and Problem Statement 4

1.3 Project Aim, Objectives, and Expected Business Value 5

1.4 Project Scope 7

1.5 Project Stakeholders and Project Management Context 7

1.6 Expected Outcomes and Benefits 8

2. PROJECT EXECUTION, MONITORING & CONTROL, AND CLOSURE 9

2.1 Introduction 9

2.2 Project Execution 9

2.2.1 Execution of the Four Functional Areas 10

2.2.2 Team Coordination and System Integration 11

2.3 Project Monitoring and Control 11

2.3.1 Progress Monitoring 12

2.3.2 Functional and Technical Control 12

2.3.3 Integration and Change Control 13

2.4 Issues, Risks, and Corrective Actions 13

2.4.1 Identified Issues and Corrective Actions 14

2.5 Testing and Verification 16

2.6 Project Closure 16

2.7 Conclusion 17

3 PROJECT PERFORMANCE DOMAINS ANALYSIS 18

3.1 Stakeholder Performance Domain 18

3.2 Team Performance Domain 19

3.3 Development Approach and Lifecycle Performance Domain 20

3.4 Planning Performance Domain 21

3.5 Project Work Performance Domain 22

3.6 Delivery Performance Domain 23

3.7 Measurement Performance Domain 24

3.8 Uncertainty Performance Domain 25

3.9 Overall Evaluation 27

4 APPLICATION OF PROJECT MANAGEMENT PRINCIPLES 28

4.1 Introduction 28
4.2 Application of the 12 Project Management Principles 28
4.3 Critical Evaluation of Project Management Application 34
5 LESSONS LEARNED 35
5.1 User Interface and Backend Compatibility 35
5.2 Challenges in Module Integration 36
5.3 Systematic Debugging and Problem Resolution 37
5.4 Appropriate Use of AI-Assisted Development Tools 38
5.5 Team Communication and Coordination 38
5.6 Continuous Testing and Integration 39
5.7 Overall Lessons and Future Improvements 40
6 FUTURE WORK 41
7 REFERENCES 43
8 APPENDICES & SUPPORTING EVIDENCE 44

1. INTRODUCTION

1.1 Project Background

The Enterprise Publishing Management System (EPMS) is a web-based information system developed for Reading Planet Publishing House as part of the IE3151 – Information Systems Project Management module at the Sri Lanka Institute of Information Technology (SLIIT). The project focuses on improving the management of publishing activities through a centralized digital platform that connects the major operational areas of a publishing house.

Publishing involves several interconnected activities, including author registration, manuscript submission, editorial review, book production, inventory management, and financial administration. These activities require accurate information exchange and coordination among authors, editors, designers, publishing staff, finance personnel, and administrators. Managing these activities through separate processes can make it difficult to maintain consistent records, monitor progress, and coordinate work efficiently.

EPMS was proposed to address these challenges by integrating the main publishing functions into a single web-based application. The system is organized around four primary functional areas: Author and Manuscript Management, Editorial Workflow and Book Production, Publishing Operations, and Finance and Royalty Management. By bringing these areas together, the project aims to support more organized workflows, improve information accessibility, and strengthen coordination between the stakeholders involved in publishing operations.

1.2 Business Context and Problem Statement

Reading Planet Publishing House faces operational challenges associated with managing publishing activities through fragmented and largely manual processes. According to the project's initial proposal, manuscript submissions were handled through separate email communications or physical copies, editorial activities were coordinated using spreadsheets, inventory records were maintained manually, and financial information was distributed across different records.

These practices created difficulties across several business functions. In manuscript management, authors had limited visibility into the progress of their submissions, while the use of separate communication channels increased the possibility of unnecessary follow-ups, delays, and

misplaced files. Within the editorial and production processes, coordination between editors, proofreaders, designers, and authors could become difficult when tasks and approvals were managed separately. This could delay the movement of manuscripts through the different production stages.

Publishing operations also required reliable inventory information. When stock records and order information were maintained separately, discrepancies could arise between recorded inventory and actual availability. In financial administration, manually calculating author royalties and compiling information from different records increased the risk of calculation errors, inconsistent information, and payment delays.

Furthermore, the separation of operational information limited management's ability to obtain a consolidated view of publishing activities and use timely information for decision-making. Unpublished manuscripts, author contracts, and financial records also required appropriate access protection because they contain confidential information and commercially valuable intellectual property.

These challenges demonstrated the need for a centralized information system capable of connecting the publishing house's major operational areas. Without a more integrated approach, the organization could continue to experience avoidable administrative work, difficulties tracking progress, inconsistent records, workflow delays, and limited visibility into business performance.

To address these issues, EPMS was proposed as an integrated web-based solution that brings manuscript management, editorial workflows, publishing operations, and financial administration into a shared platform. The business value of the proposed system lies in improving coordination, maintaining more consistent information, supporting operational transparency, strengthening financial administration, and helping management make better-informed decisions.

1.3 Project Aim, Objectives, and Expected Business Value

The primary aim of the Enterprise Publishing Management System is to provide Reading Planet Publishing House with a centralized web-based platform for managing its key publishing operations. The project aims to integrate previously fragmented workflows into a centralized

system and deliver a tested and functional Minimum Viable Product (MVP) within the planned project period.

The main objectives of EPMS are as follows:

- **Author and Manuscript Management:** Support author registration, profile management, manuscript submission, supporting-file handling, and submission-status tracking through a centralized process.
- **Editorial Workflow and Book Production:** Support the coordination of manuscript reviews, revisions, proofreading, design activities, and author approvals throughout the production process.
- **Publishing Operations:** Support publishing-related activities, including printing, inventory management, customer orders, delivery management, and distribution, in accordance with the implemented system functionality.
- **Finance and Royalty Management:** Support the management of administrative settings, financial records, invoices, payments, author royalty agreements, royalty calculations, and related reports, according to the implemented system functionality.
- **Data Management and Security:** Maintain structured data relationships and apply appropriate validation and access controls to help protect confidential information and prevent unauthorized access.
- **Project Management and Quality:** Apply project management practices throughout planning, development, integration, testing, and evaluation to support the delivery of a functional and integrated system.

The expected business value of EPMS extends beyond converting manual records into digital formats. A centralized platform is intended to improve operational transparency by making manuscript and production progress easier to monitor. Structured workflows can support coordination between departments, while consistent inventory and financial information can improve record management and operational control. Appropriate access controls can also help protect unpublished manuscripts, author contracts, and sensitive financial information.

The initial project proposal identifies the development and testing of the four functional areas as the main focus of the system. The expected benefits provide a basis for evaluating the project's

intended value; however, the actual achievement of these benefits depends on the functionality delivered and the availability of appropriate implementation and performance evidence.

1.4 Project Scope

The scope of the Enterprise Publishing Management System (EPMS) covers four main functional areas identified in the initial project proposal. Together, these areas are intended to support the publishing process, from author and manuscript management through editorial review and book production to publishing operations, financial administration, and royalty management. The extent to which each function was delivered is assessed according to the implemented system functionality.

1.5 Project Stakeholders and Project Management Context

EPMS relates to several stakeholder groups whose activities and information requirements are connected to publishing operations. Authors require a process for submitting manuscripts and following their progress. Editors, proofreaders, and designers are involved in the review and production stages. Publishing staff require organized operational and inventory information, while finance personnel work with financial records, payments, and royalty-related information. Administrators are concerned with system access, data management, and the operation of the platform.

Considering these stakeholder needs helps explain why the system is divided into interconnected functional areas rather than developed as a collection of unrelated features. Each area supports a different aspect of publishing, while the overall system requires consistent data exchange and coordination between components.

The development of EPMS also required coordination between team members responsible for different functional areas and technical components. Since the system contains interconnected features, the implementation of individual components needed to be considered alongside their relationships with the frontend, backend, database, and other modules.

During implementation, the team worked on connecting the user interface, backend services, and database to support the required functionality. The development process included addressing issues affecting form submission, file uploads, and interactions between related features. These

challenges required the team to investigate component connections, coordinate changes, and test the affected functionality. By the final demonstration, all implemented functions were working correctly, providing evidence of the progress made in resolving the issues encountered during development.

Project management activities were important for organizing tasks, communicating requirements, coordinating changes, monitoring progress, and addressing technical issues. Collaboration between team members was particularly important when changes to one feature affected another feature or when data exchanged between modules did not match the expected requirements.

1.6 Expected Outcomes and Benefits

EPMS is intended to provide Reading Planet Publishing House with a more organized approach to managing its publishing operations. By bringing major functional areas into a centralized platform, the system is expected to improve information accessibility, reduce dependence on disconnected records, and support coordination among the stakeholders involved in the publishing process.

The expected benefits include improved visibility into manuscript progress, better coordination of editorial and production activities, more consistent inventory information, improved organization of financial records, and stronger protection of confidential information through appropriate access controls. Centralized operational information can also provide a more useful basis for monitoring activities and supporting management decisions.

From a project development perspective, EPMS provided an opportunity to apply information systems project management concepts while developing and integrating a practical web-based application. The work involved requirements interpretation, frontend and backend development, database integration, testing, debugging, and team coordination. The experience gained through these activities provides a foundation for improving future software development projects and applying more structured approaches to system integration and quality assurance.

2. PROJECT EXECUTION, MONITORING & CONTROL, AND CLOSURE

2.1 Introduction

This chapter describes the execution, monitoring and control, and closure activities undertaken during the development of the Enterprise Publishing Management System (EPMS) for Reading Planet Publishing House. Building on the business context, objectives, and scope established in Chapter 1, this chapter explains how the project requirements were translated into an integrated web-based information system covering author and manuscript management, editorial workflow and book production, publishing operations, and finance and royalty management.

The chapter also discusses the coordination of development activities, integration of the system's components, identification and resolution of technical issues, management of implementation risks, and verification of the completed functionality. Finally, it examines the project outcomes, remaining limitations, and activities associated with final evaluation and closure.

2.2 Project Execution

Project execution involved translating the requirements identified during the initial analysis into functional components of the EPMS application. The development process focused on creating a centralized platform that connects the major publishing activities and addresses the problems associated with fragmented records, manual processes, limited workflow visibility, and disconnected operational information.

The project required the coordination of frontend development, backend implementation, database integration, authentication, and business workflow development. Since the four functional areas depend on related information and processes, their implementation had to be considered both individually and as part of the overall application.

The application was developed using Java and Spring Boot for backend processing, MySQL for persistent data storage, and web technologies for the user interface. The implementation follows a layered structure in which controllers receive requests, services process business rules, repositories interact with the database, and entities represent business records. This structure supports the separation of responsibilities and facilitates the maintenance and integration of different features.

2.2.1 Execution of the Four Functional Areas

Author and Manuscript Management :

The first functional area supports the management of author information and manuscript submissions. Its implementation provides the foundation for subsequent publishing activities by maintaining manuscript-related information, supporting files, and submission statuses. This functionality addresses the need for a more centralized approach to manuscript handling and improves the availability of information required by editors and other authorised users.

Editorial Workflow and Book Production :

The second functional area manages the transition from manuscript submission and editorial review to production readiness. It includes editorial assignment, review decisions, revision requests, design assignment, file uploads, author approval, production quality control, and release for printing.

The workflow supports different outcomes at important decision points. Editors can accept or reject manuscripts or request revisions. Revised manuscripts are associated with their revision records and returned for further editorial review. During production, failed quality checks require the Designer to correct and resubmit the relevant files. Author-requested design changes also require a revised design version and the appropriate review process. These controls help maintain a structured publishing workflow and reduce the risk of incomplete or outdated files progressing to later stages.

Publishing Operations :

The third functional area supports publishing-related operational activities, including inventory, orders, sales, and distribution. Its purpose is to improve the organisation and consistency of operational records and support the movement of books through the publishing and distribution process. Integration with the other functional areas is important because book information and completed sales provide information needed by downstream operational and financial activities.

Finance and Royalty Management :

The fourth functional area manages financial information associated with publishing operations. Its functionality includes relevant author agreements, sales-related information, invoices,

expenses, payments, financial reporting, and royalty calculations. Centralising these activities supports more organised financial administration and reduces dependence on disconnected records.

The four functional areas collectively address the objectives established in Chapter 1. Their integration enables the system to support a broader publishing lifecycle rather than operating as four unrelated applications.

2.2.2 Team Coordination and System Integration

The development of EPMS required coordination between team members responsible for different functional areas and technical components. Each area had specific responsibilities, but its implementation also depended on shared data, interfaces, and interactions with the rest of the application.

Integration activities involved connecting the user interface with backend services and database operations. The team also needed to ensure that related components exchanged information in the expected format and that changes to one feature did not unintentionally affect another feature.

Particular attention was required for form submissions, file uploads, and interactions between related features. These activities are important because an application may contain individually implemented components that do not function correctly when integrated. Addressing these dependencies was therefore an important part of project execution.

2.3 Project Monitoring and Control

Project monitoring and control involved assessing the implementation progress, checking whether the system behaved according to the defined requirements, and addressing problems that could affect integration or functionality. These activities were important for maintaining alignment between the original project objectives and the developed application.

Monitoring considered both feature-level progress and the behaviour of the integrated system. Completing an individual feature was not sufficient on its own; the feature also needed to interact correctly with related components and maintain the expected business information.

2.3.1 Progress Monitoring

Project progress was monitored through team meetings and discussions during the development of the Enterprise Publishing Management System (EPMS). These discussions provided opportunities for team members to review implementation progress, discuss technical difficulties, coordinate work across the four functional areas, and identify issues affecting integration.

Progress was considered in relation to the implementation of Author and Manuscript Management, Editorial Workflow and Book Production, Publishing Operations, and Finance and Royalty Management. Attention was also given to the interactions between the frontend, backend services, and database, as the functionality of the integrated application depended on these components working together.

The team encountered challenges involving module integration, form submissions, and file uploads. These issues were discussed and investigated during development to support the operation of the connected features. The final demonstration and manual checks provided an opportunity to review the implemented functionality and assess whether the demonstrated operations worked as expected.

However, progress monitoring was primarily based on team discussions, manual verification, and the final demonstration. The project did not have a formal set of documented test results or quantitative progress metrics available for this report. Consequently, the final demonstration provides evidence of the functionality demonstrated at that stage, but it does not establish that every possible operating condition or requirement was tested.

2.3.2 Functional and Technical Control

Functional control focused on checking whether the application supported the intended business processes. For example, manuscript information needed to remain associated with the correct author, editorial decisions needed to result in the appropriate workflow status, and production files needed to be available before the relevant approval and release activities could proceed.

Technical control focused on the interactions between the frontend, backend services, and database. Form submissions, file uploads, data validation, and communication between components required verification to ensure that user actions produced the expected results.

Access control was also important because the application handles unpublished manuscripts, author information, contracts, and financial records. Appropriate authentication and authorisation mechanisms help restrict access to users according to their responsibilities.

Database consistency was another control consideration. Related records needed to maintain their relationships as information moved between manuscript management, editorial processing, production, publishing operations, and finance. Validation and business rules help reduce the possibility of incomplete or inconsistent information being accepted by the system.

2.3.3 Integration and Change Control

Changes to individual components had to be considered in relation to the wider application. An adjustment to a form, backend service, database structure, or shared interface could affect another functional area that depended on the same information or behaviour.

The team therefore needed to coordinate related changes and verify affected functionality after modifications. Integration checks were particularly important when connecting frontend actions with backend operations or when data exchanged between components did not match the expected requirements.

Database changes also required attention because changes to entities or table structures could affect existing functionality. Similarly, changes to shared application components needed to be considered alongside their impact on the other functional areas.

These activities helped maintain consistency between the implemented features and the overall system requirements.

2.4 Issues, Risks, and Corrective Actions

During the development of the Enterprise Publishing Management System (EPMS), the team encountered technical challenges related to frontend and backend compatibility, module integration, form submissions, file uploads, and data exchange between interconnected features. These challenges were significant because EPMS was developed as an integrated application in which the four functional areas depended on shared information and interactions between the user interface, backend services, and database.

As discussed in Chapter 1, the purpose of EPMS was to address fragmented publishing processes through a centralized information system. Consequently, technical issues affecting the integration of its components had to be investigated and resolved to ensure that the implemented functions operated as intended.

2.4.1 Identified Issues and Corrective Actions

During development, the team encountered challenges involving frontend and backend compatibility, form submissions and file uploads, module integration, defect identification, and the effects of changes on related functionality. These issues highlighted the importance of coordinating interconnected components and verifying their behavior within the integrated system.

Table 2.1 summarizes the identified issues, their potential impact, and the responses or improvement measures described in the project report. Where a measure represents a recommendation for future development rather than a confirmed action completed during the project, it is identified accordingly.

Table 2.1: Identified Issues and Corrective Actions

Issue identified	**Impact on Issue identifiedthe project**	**Corrective action or improvement**
Frontend and backend compatibility	Features that appeared functional in the user interface did not always produce the expected results after integration with backend services.	The team examined interactions between frontend requests, backend processing, and database operations to investigate compatibility problems.
Form submission and file-upload issues	Submitted information or uploaded files could fail to produce the expected result when connected components did not handle the data correctly.	The team investigated interactions between the interface and backend. Earlier testing of actual form submissions and file uploads was identified as an

		improvement for future projects.
Module integration difficulties	Changes to one feature could affect related features, while exchanged information could fail to meet the receiving component's requirements.	The team discussed relevant changes and examined dependencies between the frontend, backend, database, and connected features.
Difficulties identifying the source of defects	Problems involving interconnected components required additional investigation and debugging.	The team examined component interactions to investigate the source of the problems. Maintaining a systematic debugging process and issue log was identified as a future improvement.
Changes affecting related functionality	Changes intended to resolve one problem could introduce defects in dependent features.	The team recognised the need to test affected functionality after changes and to verify related features that might be affected.
Insufficient early integration and testing	Problems could remain undetected when features were tested individually but not as part of the integrated system.	The team recognized the need to verify affected functionality after modifications and identified broader regression testing as an improvement for future projects.

The issues described above demonstrate the importance of early integration, coordinated changes, and verification of related functionality. The final demonstration provided an opportunity to assess the implemented functions, while more systematic integration and regression testing would provide stronger evidence of reliability across the wider application.

2.5 Testing and Verification

Testing and verification were important for assessing whether the implemented functionality met the expected requirements and whether the components operated correctly when integrated.

Verification of EPMS should consider both individual features and complete business workflows. Feature-level verification examines whether a specific operation produces the expected result, while workflow-level verification checks whether several related operations work together in the correct sequence.

For Author and Manuscript Management, relevant checks include the handling of author and manuscript information and the processing of manuscript submissions. For Editorial Workflow and Book Production, verification includes editorial decisions, revision handling, design uploads, author approval, quality control, and release for printing.

For Publishing Operations, testing should examine the consistency of operational records and the handling of inventory, orders, sales, and distribution-related information. For Finance and Royalty Management, verification should consider financial record processing, invoice and payment operations, royalty calculations, and the relevant validation rules.

Integration verification is particularly important because the functional areas share information. For example, book and sales information used in financial operations must correspond to the appropriate records in the publishing workflow.

The final demonstration provided evidence that the implemented functions were working correctly at that stage of the project. Supporting evidence for the report should include available screenshots, test cases, test results, and demonstration records. Any untested requirements or outstanding limitations should be documented separately rather than assumed to be verified.

2.6 Project Closure

Project closure involved reviewing the developed application, assessing the completion of the intended functionality, and preparing the system and its documentation for final demonstration and evaluation.

The closure review considered EPMS as an integrated publishing information system covering the four functional areas described in Chapter 1. The purpose was to determine whether the

implemented features collectively supported the intended publishing processes and addressed the problems identified in the initial business context.

The final demonstration was an important part of this review because it allowed the team to show the operation of the implemented functions and confirm that the issues affecting the demonstrated functionality had been addressed.

The project deliverables included the application source code, implemented frontend and backend components, database-related artefacts, workflow diagrams, and project documentation. These materials provide a basis for examining the system structure, understanding the business processes, and reviewing the implementation.

Closure also requires distinguishing between completed functionality and areas that may require future improvement. Although the final demonstration showed the implemented functions working correctly, further improvements may still be considered in areas such as broader testing, performance evaluation, deployment preparation, and ongoing maintenance.

Any unresolved limitations should be recorded clearly so that the final evaluation reflects the actual state of the application. This ensures that project completion is assessed using available evidence rather than assumptions about functionality that has not been verified.

2.7 Conclusion

This chapter examined the execution, monitoring and control, and closure activities associated with the development of EPMS for Reading Planet Publishing House. Project execution focused on implementing the four functional areas and integrating the frontend, backend, and database components into a centralized publishing management application.

Monitoring and control focused on implementation progress, workflow correctness, data consistency, access restrictions, and the identification of technical issues. Problems involving form submissions, file uploads, and interactions between related features required investigation, coordination, and verification of the affected functionality. The final demonstration provided evidence of the progress made in resolving these issues and delivering the implemented functions.

Overall, the project applied software development and project management practices to address the need for a more integrated approach to publishing operations. The completed implementation

provides a foundation for managing author and manuscript information, editorial and production workflows, publishing operations, and financial and royalty-related activities within a single system. The final evaluation of the project should consider the implemented functionality alongside the available testing evidence, documentation, and any remaining limitations.

3 PROJECT PERFORMANCE DOMAINS ANALYSIS

3.1 Stakeholder Performance Domain

Stakeholder engagement played an important role in the EPMS project because the system needed to satisfy the client's business requirements. The team communicated with the client through Zoom meetings, demonstrated completed functionality incrementally, and obtained feedback throughout development. This approach enabled the client to review the system before the entire solution was completed, allowing the team to validate whether the developed features aligned with the agreed expectations.

The client's positive feedback and formal approval of the completed system indicate that the stakeholder engagement approach supported an acceptable project outcome. Furthermore, no requirement or feature changes were reported during development. This helped the team maintain the agreed scope and concentrate on implementation rather than repeatedly adjusting planned functionality. However, the absence of reported changes does not necessarily demonstrate that stakeholder engagement was fully effective, as it may also reflect the stability of the original requirements. Therefore, the outcome should be evaluated alongside the team's ongoing communication and validation activities.

A limitation of the approach is that effective communication does not automatically ensure that stakeholder expectations and decisions are consistently documented. Without formal records of feedback, requirement validation, and approval decisions, it may become difficult to trace how particular requirements were interpreted and confirmed. Although the client approved the completed system, stronger documentation would provide a clearer basis for demonstrating how stakeholder expectations were managed throughout development.

Overall, stakeholder engagement contributed positively to the project by maintaining client involvement and supporting acceptance of the delivered solution. Future projects could strengthen this performance domain by maintaining a requirement validation record, documenting client

feedback, and linking approval decisions to the corresponding requirements. These practices would improve traceability, reduce the possibility of misunderstandings, and provide a stronger basis for evaluating stakeholder satisfaction.

3.2 Team Performance Domain

The EPMS project was developed by four team members who collaborated to implement and integrate the system's four main epics. Following requirements gathering and analysis, the team divided the work into epics and distributed tasks equally among the members. Development activities were conducted in parallel, while physical meetings, telephone calls, and Zoom meetings supported communication and coordination. GitHub was used to share code and integrate individual contributions.

One strength of the team's approach was its collaborative response to technical difficulties. When integration problems occurred, members discussed the issues, resolved merge conflicts, aligned database tables, and supported one another in solving technical problems. This demonstrates that teamwork extended beyond completing individually assigned tasks to include shared responsibility for producing a functioning system. The successful integration of all four epics suggests that this collaboration contributed to achieving the project's overall objective.

However, the team experienced merge conflicts, differences in code structure, and database mismatches. These challenges reveal a weakness in technical coordination across independently developed components. Although equal task allocation provided a balanced distribution of responsibilities, it did not necessarily ensure that members followed compatible implementation decisions. This distinction is important because a project depends not only on individual productivity but also on how effectively individual contributions work together.

The integration difficulties also suggest that the team could have established clearer shared technical practices before development progressed significantly. Agreeing on coding conventions, database structures, and component interfaces earlier would have reduced the likelihood of inconsistencies. In addition, regular integration checks could have identified incompatibilities before multiple components depended on them.

Therefore, the team demonstrated effective collaboration and problem-solving, but its experience also shows that cooperation alone cannot fully compensate for insufficient technical alignment. Future projects should combine collaborative communication with shared development standards, clearly defined integration responsibilities, and frequent reviews of cross-component compatibility. This would improve coordination and reduce the time required to resolve integration problems.

3.3 Development Approach and Lifecycle Performance Domain

The team followed an Agile Scrum approach and completed four sprints. The development lifecycle included gathering and analysing client requirements, organising requirements into epics and tasks, implementing functionality incrementally, demonstrating completed features to the client, testing the system, and integrating the developed components.

A key strength of this approach was incremental development. Rather than postponing all client feedback until the end of the project, the team demonstrated individual parts of the system during development. This created opportunities to validate functionality against client expectations and identify defects before final completion. Manual testing was conducted on individual features and the integrated system, while identified bugs were fixed before completion. These activities supported iterative improvement and helped the team produce a solution that received formal client approval.

Nevertheless, the project experience indicates that following an Agile Scrum approach does not automatically guarantee effective integration. Because members developed separate epics in parallel, differences in code structure and database design created compatibility problems when the components were combined. This suggests that the team applied incremental development at the feature level but could have strengthened coordination at the system level. Individual components may function independently while still creating problems when combined into a complete solution.

The team also reported no requirement or feature changes during development. While this supported scope stability, it does not independently establish the effectiveness of the development approach. The stronger indicators are the combination of incremental demonstrations, testing, completed sprints, integration of the four epics, and client acceptance.

Overall, the chosen approach supported incremental delivery and client involvement, but the integration difficulties exposed opportunities to improve its implementation. Future projects should incorporate regular cross-epic integration, shared technical standards, and integration testing within each sprint. These practices would align the development of individual features with the continuous validation of the complete system, reducing late-stage compatibility problems and improving delivery confidence.

3.4 Planning Performance Domain

Project planning began with requirements gathering and analysis. The team organized the requirements into four main epics and divided them into smaller tasks, which were allocated equally among the four members. The project followed an academic schedule covering Weeks 05 to 15, including proposal preparation, Sprint 0, subsequent development sprints, and the final presentation and exhibition.

Progress was monitored through GitHub commits, the project board, regular team discussions, and checks of completed functionality. The team reported completing all four sprints on time and finishing the project within the planned period. These outcomes suggest that the schedule and task allocation approach provided a workable structure for organizing development activities and maintaining progress toward the final deadline.

However, the integration difficulties reveal a limitation in how technical dependencies were accommodated within the plan. Merge conflicts, differences in code structure, and database mismatches required additional coordination after individual components had been developed. These problems suggest that the plan could have given greater consideration to the relationships between tasks and the risks associated with combining independently developed components.

Another limitation is that equal task allocation does not necessarily represent an optimal distribution of work. Tasks may differ in complexity, technical uncertainty, and the number of dependencies they have on other components. Consequently, allocating an equal number of tasks or an equivalent workload does not guarantee that every member faces the same level of effort or that integration responsibilities are adequately covered.

Although the project was completed on schedule, this achievement should not obscure the opportunity to make planning more responsive to technical complexity. Future projects should identify cross-epic dependencies during planning, assess task complexity alongside workload, and allocate specific time for integration and regression testing. Reviewing these dependencies during sprint planning would also help the team anticipate potential conflicts instead of responding only after they occur.

Therefore, the planning approach was effective in maintaining the project schedule, but the integration challenges indicate that greater emphasis on dependency management and technical risk would have made the plan more robust. A stronger plan would balance schedule control with sufficient flexibility to address integration problems without compromising the quality of the final system.

3.5 Project Work Performance Domain

Project work involved implementing the planned functionality, coordinating parallel development, monitoring tasks, testing features, and integrating the contributions of all four team members. GitHub supported code sharing and integration, while the project board and regular discussions helped the team monitor progress and identify outstanding work.

The team encountered several technical difficulties when combining independently developed components, including merge conflicts, differences in code structure, and database mismatches. Members addressed these problems through discussion, conflict resolution, database alignment, and collaborative troubleshooting. Bugs identified during testing were also fixed before project completion. These activities demonstrate that successful project work required continuous coordination and technical problem-solving in addition to implementing assigned functionality.

A major strength was the team's willingness to resolve shared problems collaboratively. Instead of treating integration issues solely as individual development problems, the team worked together to achieve a functioning system. This contributed to the integration of all four epics and supported the project's successful completion.

However, the recurring integration difficulties indicate that some technical decisions were not sufficiently aligned before components were combined. This created additional work that could

potentially have been reduced through earlier agreement on shared database definitions, component interfaces, and coding conventions. The experience illustrates an important distinction between completing individual tasks and managing the dependencies that determine whether the complete system functions consistently.

The team could also have reduced integration-related rework by reviewing code before merging and integrating components more frequently throughout development. These practices would allow incompatibilities to be identified closer to the point at which they are introduced, making them easier to investigate and correct.

Overall, project work was successful in producing and integrating the planned functionality, but the experience revealed opportunities to improve technical coordination and prevent avoidable rework. Future projects should treat integration as a continuous development activity rather than primarily a problem to resolve when independently developed components are combined. This would support more efficient collaboration and allow greater attention to be given to complete-system testing and quality.

3.6 Delivery Performance Domain

The main delivery outcomes were the completion of the EPMS system, integration of all four epics, completion of the four sprints within the reported schedule, and formal client approval. The team also demonstrated functionality incrementally during development, allowing the client to review the solution before final completion. Manual testing covered individual features and the integrated system, and identified bugs were fixed before delivery.

These outcomes indicate that the team successfully converted the planned development activities into a completed solution that received client acceptance. Incremental demonstrations and ongoing validation also reduced reliance on a single final presentation as the only opportunity for client review. The combination of implementation, testing, integration, and approval provides a stronger basis for evaluating delivery success than schedule completion alone.

Nevertheless, delivery success should be distinguished from product quality and long-term business value. Completing the system within the planned period demonstrates schedule performance, while formal client approval indicates acceptance of the delivered solution. Neither

outcome independently proves that the system will achieve its intended benefits during operational use. The available project information does not establish post-deployment outcomes such as user adoption, processing-time reductions, or long-term reliability.

The integration challenges also highlight the importance of evaluating delivery in terms of both completion and readiness. Although the team resolved the reported technical problems and integrated all four epics, a more systematic record of acceptance criteria and test outcomes would make it easier to demonstrate that the delivered functionality met defined quality expectations.

Future projects should therefore use a final acceptance checklist linked to the agreed requirements, document testing results, and record any known limitations at handover. If the system is subsequently used in an operational environment, user feedback and appropriate usage measures could help determine whether the expected business benefits have been achieved.

In conclusion, the project demonstrated successful delivery in terms of completion, integration, schedule, and client approval. However, a comprehensive evaluation should recognise that these outcomes establish reported delivery success rather than verified long-term operational effectiveness. Distinguishing these dimensions would provide a more balanced assessment of project performance.

3.7 Measurement Performance Domain

The team monitored project progress through GitHub commits, the project board, regular discussions, and checks of completed functionality. These activities provided practical ways to observe development progress and coordinate work. Completion of all four sprints within the planned schedule also indicates that the team achieved its reported scheduling objectives.

Manual testing provided another mechanism for evaluating system functionality. The team tested individual features and the integrated system, identified bugs, and fixed them before project completion. Incremental client demonstrations and formal approval provided additional indications that the implemented functionality was reviewed against stakeholder expectations.

However, monitoring activities should be distinguished from systematic performance measurement. GitHub commits and project-board updates can show that development work is

taking place, but they do not automatically demonstrate the quality, complexity, or business value of that work. Similarly, completing sprints on time does not establish whether the team used its resources efficiently or whether the system achieved all intended quality objectives.

The exact performance metrics and measurement records used by the team could not be recalled. Therefore, numerical results such as defect counts, productivity rates, test pass percentages, and requirement coverage cannot be reported reliably. This limits the extent to which project performance can be evaluated objectively. Although the available information supports conclusions about reported schedule completion, testing activities, and client approval, it does not support quantitative claims about overall efficiency or quality.

This limitation matters because effective measurement should help a team identify performance trends, compare actual outcomes with expectations, and make informed decisions. Without consistently recorded indicators, the team may recognise that problems occurred but have limited ability to determine their frequency, severity, or impact. Consequently, some improvement opportunities may remain based on general experience rather than measurable results.

Future projects should define a small set of relevant performance indicators during planning and record them consistently throughout development. Suitable indicators could include planned versus completed tasks, test cases executed and passed, defects identified and resolved, and requirement coverage. These measures should be interpreted in context rather than treated as isolated indicators of success.

Overall, the team used practical progress-monitoring and testing activities, but the lack of recalled quantitative records limits the depth of performance evaluation. Establishing clear measurement criteria early would help future teams move beyond confirming completion toward understanding the efficiency, quality, and effectiveness of their project work.

3.8 Uncertainty Performance Domain

The EPMS project involved technical uncertainty because four separately developed epics needed to operate together as one integrated system. The team experienced merge conflicts, differences in code structure, and database mismatches during integration. These problems illustrate how parallel

development can introduce uncertainty when individual components depend on shared data structures and compatible implementation decisions.

The team reported identifying risks and discussing ways to address them. When integration problems occurred, members communicated with the relevant developers, resolved merge conflicts, aligned database tables, and supported one another in solving technical issues. Manual testing of individual features and the integrated system also helped identify defects before completion. These responses demonstrate the team's ability to react to encountered technical problems and work toward a functioning solution.

However, responding to problems after they occur is not the same as proactively managing uncertainty. The integration difficulties suggest that some compatibility risks became apparent during integration rather than being fully addressed before or during parallel development. Earlier agreement on database structures, component interfaces, and coding standards could have reduced the likelihood of these problems or made them easier to detect.

The team ultimately completed the project within the reported schedule, integrated all four epics, and obtained formal client approval. These outcomes indicate that the encountered difficulties were addressed without reported delays to the sprint schedule. Nevertheless, the absence of schedule delays does not mean that the risks had no impact: integration problems still required additional coordination and corrective work.

The available information does not establish whether the team maintained a formal risk register, assigned risk owners, or assessed the likelihood and impact of individual risks. Therefore, the project can be evaluated in terms of its reported responses to technical uncertainty, but it cannot be concluded that a formal risk-management process was implemented.

Future projects should identify integration risks during planning, assess their likelihood and potential impact, assign responsibility for monitoring them, and review them throughout each sprint. Regular integration testing shared technical standards, and clear responsibility for cross-epic dependencies would also improve the team's ability to detect and address problems early.

Overall, the team demonstrated adaptability and collaborative problem-solving when technical uncertainty materialised. However, the experience also highlights the difference between reacting

effectively to problems and proactively reducing their likelihood. Strengthening early risk identification and continuous integration would help future projects manage uncertainty more systematically and reduce the possibility of avoidable rework.

3.9 Overall Evaluation

Overall, the EPMS project demonstrated effective client engagement, collaborative teamwork, structured planning, incremental development, and delivery within the reported schedule. The integration of all four epics and formal client approval indicate that the team achieved its main delivery objective. The team's response to merge conflicts, code-structure differences, and database mismatches also demonstrates its ability to resolve technical challenges through collaboration.

However, the analysis identifies several limitations that should be considered when evaluating the project's overall effectiveness. Equal task allocation did not eliminate coordination difficulties, incremental development did not automatically ensure smooth integration, and schedule completion did not establish long-term operational success. In addition, the absence of recalled quantitative performance metrics limits the extent to which development efficiency and system quality can be evaluated objectively. These limitations demonstrate that successful completion is only one dimension of project performance.

The key lesson is that the performance domains are interdependent. Planning must account for technical dependencies, teamwork must support shared implementation decisions, development activities must include regular integration, measurement must provide meaningful information about progress and quality, and uncertainty management must address risks before they become significant problems. Improvements in these areas could reinforce one another rather than functioning as isolated changes.

Therefore, the EPMS project can be considered successful in its reported completion and client-acceptance outcomes, while still presenting opportunities for improvement in integration planning, technical coordination, measurement, and proactive risk management. A more systematic approach to these areas would strengthen future project execution and provide a more objective

basis for evaluating whether project outcomes meet both immediate delivery expectations and longer-term quality objectives.

4 APPLICATION OF PROJECT MANAGEMENT PRINCIPLES

4.1 Introduction

The Enterprise Publishing Management System (EPMS) project is a project that will use project management concepts to plan, develop, coordinate and evaluate an integrated web-based publishing management solution. The project will be aimed at having a single repository for author and manuscript data, editorial workflows, book production, inventory, customer orders, financial records and royalty calculations, replacing the piecemeal use of manual systems.

Project management principles are used to ensure the team keeps the project goals, assigns responsibilities, handles project risks, maintains project quality and produces a functional (MVP) within the project timeline. The following sections examine the relevance of the 12 principles of project management to the EPMS project, along with the benefits, challenges and implications of applying these principles to the success of the EPMS project.

The Twelve Project Management Principles are applied to this project.This project is applying the Twelve Project Management Principles.

4.2 Application of the 12 Project Management Principles

1) Stewardship

Stewardship means to manage project resources and information responsibly, ethically and transparently.

Project stewardship is captured in the planning of the project needs, project development, financial information, and author information in the EPMS project. The system proposed includes authentication and role-based access control to ensure the security of the manuscripts and financial records. The project budget also also segregates estimated commercial development expenditure versus actual expenditure, which will facilitate clarity in financial planning.

But securing data confidentiality, access and proper financial calculations demands constant attention during development and testing.

2) Team

The team principle stresses the teamwork, the responsibility sharing and communication between team members.

The EPMS project has 4 members: Scrum Master and 3 team members. The system consists of four epics: Author/ Manuscript Management, Editorial Workflow and Book Production, Publishing Operations, Administration/ Finance and Royalty Management. This modular design promotes the distribution of development work obligations and enables team members to deal with specific business capabilities.

The modules are interdependent on information and therefore collaboration is especially critical. For instance, sales data in epic 4 needs to be completed in epic 3 and author info and contract info needs to be completed in epic 1. Often development of modules is done separately and communication, code reviews and integration testing are necessary to avoid any integration problems.

3) Stakeholders

Engagement of stakeholders makes sure that the system meets the needs and expectations of the intended users.

The EPMS project considers authors, editors, designer, production staff, warehouse, customers, sales manager, financial officer, administrator and executives as significant stakeholders. They are expressed using user stories in the product backlog.

The authors, for instance, require to trace the progress of their manuscript and look over royalty statements, and customers require to browse books, place orders and check on deliveries. Financial personnel need to keep track of revenues accurately and calculate royalty amounts.

When the needs of the stakeholders are taken into account, it's easier for the team to create relevant functionality. But if there's a gap in expectations for stakeholders, feedback and acceptance testing will be required to ensure that the features which were implemented meet the intended user base.

4) Value

The value principle is aimed at providing value added and not just at completing technical tasks.

The most significant advantage of EPMS lies in the fact that the publishing-house functions are gathered in a single place. The purpose of the system is to decrease the amount of manual labor, enhance the visibility of the information, reduce calculation errors, and aid quicker decision-making.

For instance, automated royalty calculations can save the time used in manually calculating author payments. Likewise, inventory control and customer ordering can aid in making the inventory of books more visible.

The project is driven by user stories and epics and focuses on the core business workflows. However, do not assume that all the benefits of the system have already been realized, but rather test the system, gather user feedback, process accuracy and compare with previous manual processes.

5) Systems Thinking

Systems thinking is about the project not being broken down into its individual parts.

EPMS is an example of this in four business areas. The book undergoes manuscript handling, editorial and production processes, publishing, and financial processes.

An example of this is completed sales data that are written into Epic 3 are used to monitor revenue and calculate royalties in Epic 4. These financial processes are also supported by author and contract information from Epic 1 as well as book publication information from Epic 2.

The dependency structure can cause the failure or delay of one module to impact other modules. Hence, integration testing, relationships in the database, and interfaces are important to ensure that the entire system is reliable.

6) Leadership

Leadership is about managing people, clarifying project goals, problem solving, and holding people accountable.

The EPMS project designates a Scrum Master to be accountable for helping with team coordination and project operations. Epic responsibilities help define the team members' roles and responsibilities.

Leadership is especially crucial when the team experiences technical problems, integration problems, or academic conflicts. Timely problem solving and effective communication ensures continued movement towards project goals.

Successful leadership should not however be judged by the allocation of roles but by the team's performance in effectively coordinating, making decisions and solving problems.

7) Tailoring

Tailoring refers to adapting project management practices to the project's size, complexity, needs and resources.

EPMS is based on an Agile approach with epics, user stories, story points, estimated hours, priorities and sprint planning. The practices are appropriate for a four member academic development team as they can be split into small functional areas.

The project also identifies Minimum Viable Product and prioritizes the key workflows such as manuscript submission, editorial review, inventory management, customer ordering, and calculating royalties.

This helps the team stay on track with core needs and within time constraints. If the complexity or dependencies of the technical solution impact the original scope and/or priority, then sprint scope and priority need to be adjusted.

8) Quality

Quality guarantees that the system developed fulfills the requirements and functions according to the desired standard.

The EPMS Definition of Done provides the following criteria for functionality: it meets agreed requirements, it is tested and reviewed, it integrates well with the system and database, and it resolves known defects. Security and role-based access should be used as appropriate.

In the realm of inventory management, customer order processing, generating invoices, and calculating royalties, the stakes are high, and errors in data can result in financial losses or operational mistakes.

For instance, the eligible completed sales and agreement rate should be used for calculating royalties. Validation, exception handling, database verification, and testing with different scenarios can help to find out defects before the acceptance.

The most difficult aspect is to make sure that the quality checks are applied to the interaction of all four epics, as well as the individual epics.

9) Complexity

The "complexity" comes from needs that are interrelated, varying expectations and interests of stakeholders, technical issues, and evolving business realities.

These are all inter-related components of EPMS, with various workflows and permissions. The system should be able to link manuscript status, production status, approval for production, book inventory, book sales, financial transactions, and authorization payments to authors.

For instance, royalty calculations rely on accurate sales records, existing royalty agreements, and valid book and author data. A record that is missing or incorrect will make it impossible to get the correct answer to a calculation.

This is managed by the modular design and separation into epics. However, there are cross-module dependencies, inconsistent data, and integration issues that still have to be addressed systematically and tested.

10) Risk

Risk Management is the process of recognizing uncertainties that may impact project goals and making plans to deal with those risks.

Some of the risks involved in the EPMS project may be delays in development, integration, database discrepancies, security issues, poor royalty calculations and defective testing.

The team can mitigate these risks by implementing early integration, performing regular database backups, using version control systems like Git and GitHub, having access control for users based on their roles, and testing out critical business scenarios.

Duplicate payment of royalties, for instance, can be minimized by testing to see if duplicate royalty calculations are being avoided. Likewise, checking for stock availability post-customer orders can minimize the chances of having the wrong inventory.

Risk management should be an ongoing process throughout the project, since new risks could arise when the project is implemented and evaluated at the end.

11) Adaptability and Resilience

Adaptability and resilience require responding to the changing circumstances and continuing along the path towards goals of the project.

The EPMS project has a modular structure and a prioritized product backlog, so the team can check the requirements, and adjust the development priorities if it is needed.

If integration with publishing operations and finance is not proceeding as planned, for instance, the team could reorder the tasks, resolve important constraints, and dedicate time for further testing.

Having a version control system, documentation of decisions made, and testing of code before final integration can also help in case of development issues.

Adjustability should not result in out-of-control scope changes in the project, however. Changes need to be evaluated in terms of time, technical relations, quality and agreed MVP.

12) Change

Projects and the environment in which they operate can change over the lifetime of a project.

The aim of EPMS is to replace the current manual workflow in the publishing house with centralized digital processes. The system also has flexible administrative settings, book categories, genres and announcements to meet changing operational needs.

For instance, configurable tax rates and invoice prefixes can ensure that administrators can keep up to date with relevant financial settings as the business evolves. There are additional reporting capabilities or integrations that may be added in the future.

To implement change management, the solution involves assessing the change, updating the specifications and documents, testing the affected modules, and notifying the other team members and interested parties of the changes. This minimises the risk of any unintended impacts to current functions.

4.3 Critical Evaluation of Project Management Application

EPMS can be developed based on project management principles. The four epics enable task distribution, and the product backlog structures the user stories based on the priority, effort and business value. A Definition of Done can be used to measure functionality and integration quality.

Key strengths of the project are the modular development, the clear definition of user stories and the attention to interconnected business workflows. The features facilitate the organization of a complex system into smaller sections and identify the interaction of modules.

But a number of obstacles must be addressed. Data structures or missing dependencies can cause issues with integration between modules. Limited time can also restrict testing, feedback from stakeholders, and documentation. In addition, giving tasks does not ensure good teamwork or delivery.

The team should review progress regularly, have open communication, test interfaces of modules early, prioritize high risk functions, and check completed work against Definition of Done to help alleviate these challenges. Evidence of project performance should be based on test results, user stories completed, defects resolved, etc. and successful end-to-end workflows.

In summary, the 12 project management principles guide the development of a project to deliver business value to its stakeholders and to meet its technical requirements.

5 LESSONS LEARNED

Working on the Reading Planet Enterprise Publishing Management System (EPMS) was an excellent opportunity for project frontend and backend compatibility, module integration, debugging, technical decision making, and team coordination. The system was made up of a few interdependent functional areas, and the development process needed to be addressed not just on a feature-by-feature basis, but on an interactional basis between the features, backend, database, and other modules.

The problems that arose during development showed the need for a systematic approach to identifying technical problems, communicating changes within the development team, and testing the application as a whole system. The team has proceeded with the affected functionality with some compatibility/integration problems that needed further investigation but have been able to show the system's implemented functions during the final demonstration.

5.1 User Interface and Backend Compatibility

One of the significant challenges was getting the features working correctly on the front-end when connecting the back end. A feature might function in the frontend but won't do so as expected after connection. Another issue was the completion of forms and upload of files; just because you see something on the user interface doesn't mean it is working correctly.

These issues brought up the issue of compatibility between the frontend, backend, and database. All the following are necessary to ensure the function works correctly: Correct handling of the requests, Correct Data Structures, Correct backend processing, Correct Database operations. While individual components appear correct, there may be a lack of communication between them, and this can impact the overall functionality.

The team examined the relationship between the front end, back end, and database to determine these issues. This was useful to focus on interaction aspects instead of focusing on the front-end part solely.

The takeaway from the main was that the development of the interface and integration of the backend should go hand in hand. It is important that team members have agreed to certain expectations before they begin implementing features that depend on other people's APIs, such as

field names, request types, handling of files, etc. Whenever you upload files or submit forms, you need to test the uploads and forms in the backend as early as possible. This would help in detecting these issues of compatibility early and hence reduce the amount of debugging during later development.

5.2 Challenges in Module Integration

The other major challenge was the integration of different aspects of EPMS. Sometimes changes to one feature impacted other features and sometimes the data from one module did not always meet the need for another module. These problems demonstrated the interdependence of the functional parts of a system and the need for consistency among the parts.

One of the main causes of these problems has been the dependency of the features, and the need for data to be compatible with the features and features to be coordinated and implemented. If a feature is modified, another feature that depends on the same information or function may be impacted as well. Similarly, when the assumptions on the data being shared vary across modules, more research and fine tuning would be required for them to get along.

These integration issues resulted in more time being taken in the identification and resolution of mistakes. They also pointed out that there is no guarantee that a feature that works when tested alone will work when it is used with the system as a whole. So, integration was regarded as a development activity, rather than a product, after completing each feature.

To address these issues, they examined the relationship between the front end and back end and database and discussed the modifications with the other team members and how they can be coordinated. Such practices enabled the team to explore how the components are related and to discuss with each other when changes might impact other parts of the system.

Next time it would be helpful for the team to pick out common data structures, data relationships, and interface assumptions prior to building dependent modules. They should be incrementally integrated and then tested for integration of related features rather than integrated and then tested. The team should also record integration problems (features affected, problem discovered, actions taken to resolve the problem). This will help in identifying the same issues and coordination, and in avoiding unneeded debugging.

5.3 Systematic Debugging and Problem Resolution

The problems with the ease of compatibility and integration of modules proved to be critical to the process of systematic debugging shown in the interface. This could be due to an issue with the connection between the front end, back end and database, or between related features if they were not working as they should. Therefore, it's important to consider the interaction between the components involved in order to solve the issue.

One of the techniques used to look at these issues was to look at the relationship between the frontend, backend, and the database. Additionally, if changes or problems were to happen across the interconnected features, there was assistance that could be provided through coordinated discussions amongst members of the team. All these were intended to further investigate and resolve the issues they encountered during the development process.

The experience demonstrated that debugging may take more time if the relationships between components in the system are not clear enough and/or if a change in one component has an unexpected impact on another feature. It also strengthened the need to test after change as opposed to taking a change for granted that it has solved a problem.

The team has been working on compatibility and integration issues, and in the final demonstration, all the functions implemented by the team were successfully demonstrated. This provided them with a useful confirmation that they had resolved the issues which have affected the demonstrated functionality. But the successful demonstration does not imply that there were no defects or problems left to be resolved.

The team should follow a systematic method of debugging their project in the future. This should include reproducing the problem, identifying the parts, checking any requests or database operations that might be relevant, making a specific change, and then re-testing the specific functionality that was involved. After the above is addressed, other features that could have a relationship with the above should be addressed to ensure that the change does not adversely affect the current functionality. Having a simple issue log would also aid in systematic investigation and help decrease repeated troubleshooting.

5.4 Appropriate Use of AI-Assisted Development Tools

The use of AI supported development tools helped to resolve technical problem solving. The recommendations made by these tools, however, were not always in line with the current implementation and the project needs. So, it would not be possible to immediately consider a suggested result to be the most appropriate answer.

This experience had increased awareness of the need to think about solutions that could be added to the current code base. Seemingly correct solutions on paper may not blend with the existing frontend, backend, database structure or similar features. If these dependencies are not evaluated prior to implementing the suggestions, then there may be more inconsistencies or errors to encounter.

The team could look at each of the suggestions the AI makes and decide whether it was identical to the one they had in place or if it was suitable for them. This re-emphasized the need to apply technical judgment and not solely automated recommendations.

In future projects, the use of AI as an aid for the developers rather than replacing the developers' understanding should be promoted. Any suggested modifications should be compared to project requirements, tested for compatibility with existing components, and tested prior to incorporation. Before making the change, the developer should have an understanding of the goal and potential effect of the proposed change. This would enable the team to make use of AI responsibly with appropriate control of the system's quality and reliability.

5.5 Team Communication and Coordination

The integration challenges highlighted the need for communication amongst those team members who are accountable for interconnected features being integrated. New features added to one module may affect another, and inconsistencies in the information coming from different modules can result in additional integration efforts. These situations made it clear that it was important to be able to communicate technical changes and dependencies clearly.

To improve coordination, one of the strategies used was to discuss changes with the team members. These talks were helpful because it was sometimes not enough to solve any integration problem – sometimes it was necessary to understand how the components involved should work together.

The outcome was that communication should not be limited to when the challenge is hard to solve but include the process of implementation and integration as well. In future projects it would be good to establish regular technical talks, define ownership of features, develop a common interface/data model and document changes which could affect other features. Early reporting of blockers would also give the appropriate team members a chance to investigate issues before they pose additional integration issues.

5.6 Continuous Testing and Integration

The difficulties faced during development showed that testing each of the features individually is not sufficient to ensure the entire system will work properly. Things that seem to be working perfectly on the frontend when they are working perfectly on the backend, and changes that work on one module that is used in another module, might affect the other module.

The experiences demonstrated that testing at multiple levels was important. Individual components need to be checked to ensure they are expected to behave as intended, and integration testing should ensure communication between the front-end, back-end, database, and other modules. In addition, end-to-end testing should be used to investigate end-to-end workflows from interaction with users to the expected behaviors of the system.

In the last demonstration, we got a good opportunity to verify that the functions we developed function properly in the example system we demonstrated. This highlighted the need for testing the full functionality as opposed to testing each component individually.

For future projects, testing should commence when the individual components are available, and then also include the integration of the components. There should be a checklist of tests for important functions such as form submission, uploading and downloading files, accessing and updating data, and testing interactions between interdependent functions. After changing a feature, the team should re-test that changed function and any other features that may be affected.

This would enable detecting defects at an Early stage, decrease repeated debugging and give more confidence to the integrated system behaves as desired.

5.7 Overall Lessons and Future Improvements

Overall, it was a great success to develop Reading Planet EPMS, and it has shown that successful software development is not only about implementing individual features. The smooth flow of data, modules and testing, and the right use of development tools and team communication guarantee that the application is reliable.

The most frequently reported challenges during the development process were some features developed on the front end were not easily combined with the back end; some features changed between the front end and back end; and data flow between front end and back end was inconsistent. These were some other problems that needed to be investigated and debugged. Examinations of interfaces between system elements and communication of change to team members proved to be beneficial strategies to deal with the issues. The functions implemented during the final demonstration were also successful, and it was demonstrated how crucial it is to keep advancing the system by continuing to investigate, fine-tune and test it.

One other improvement to be made in the development process for future projects would be to have a system of version control as soon as the development process begins. A shared git repository and branching system would be useful to coordinate changes to code, maintain history of changes and maintain work on various features. The team may create a shared repository, agree on a naming convention for branches, commit the changes often, and review the changes prior to merging the changes to the main branch. This would enhance change tracking, collaboration, and give a better structure for codebase management as the system expands.

Shared technical requirement definition should also be done early for future projects, features related to the requirement should be added bit by bit, forms and file uploads should be checked with real backend functionality, and there should be a systematic way to debugging and regression testing. Technical changes need to be communicated, and suggestions made by AI ought to be checked and validated prior to use.

The enhancements made would help in early detection of issues, minimization of unnecessary rework, teamwork, and in the manufacture of a consistent and reliable integrated system. The lessons learnt in EPMS serve as the hands-on basis to improve the team's future software development process.

6 FUTURE WORK

The Enterprise Publishing Management System (EPMS) provides a basis to digitalize the key parts of the publishing house, but more improvements can be taken into account for further development. Recommendations are made that are related to limitations, operation, and ways to enhance the long-term value of the system.

1) Advanced Reporting and Business Analytics

Future versions could introduce more advanced dashboards, customizable reports, and visual analytics for sales trends, inventory performance, expenses, profits, and royalty liabilities. These features would help executives identify best-selling books, monitor financial performance, and make evidence-based business decisions.

2) Enhanced Security and Data Protection

The system could be strengthened through additional security measures, including multi-factor authentication, more detailed audit logs, automated backup procedures, and regular security assessments. These improvements would help protect confidential author information, manuscripts, customer details, and financial records.

3) Improved Online Store Experience

The customer-facing store could be enhanced with book recommendations, customer reviews, wish lists, advanced search filters, and improved mobile responsiveness. Integration with secure online payment gateways and automated customer notifications could also make purchasing and order tracking more convenient.

4) Automated Notifications and Workflow Improvements

Automated email or in-system notifications could inform authors about manuscript status changes, revision deadlines, design approvals, royalty statements, and payment updates. Staff could receive alerts for low stock, delayed production tasks, and pending approvals, helping reduce manual follow-ups and communication delays.

5) Integration and Performance Optimization

Future development could improve integration between the four epics through more comprehensive API testing, consistent data validation, and automated end-to-end testing. Database optimization, performance testing, and scalable hosting could also help the system handle larger numbers of books, customers, transactions, and users.

6) Continuous User Feedback and Maintenance

After deployment, the system should be evaluated using feedback from authors, publishing staff, customers, finance officers, and administrators. Identified usability issues and defects could be prioritized for subsequent development cycles. Regular maintenance, updated documentation, and regression testing would help maintain system reliability as new features are introduced.

Conclusion

The proposed future improvements would extend EPMS beyond its initial Minimum Viable Product by strengthening security, usability, reporting, automation, and scalability. These enhancements should be prioritized according to stakeholder feedback, business value, technical dependencies, available resources, and project risks. A phased implementation approach would allow the publishing house to improve the system gradually while maintaining the reliability of its existing operations.

7 REFERENCES

[1] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 7th ed. Newtown Square, PA, USA: Project Management Institute, 2021.

[2] K. Schwaber and J. Sutherland, *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*, Nov. 2020. [Online]. Available: https://scrumguides.org/scrum-guide.html. [Accessed: Oct. 20, 2026].

[3] K. Beck *et al*., "Manifesto for Agile Software Development," 2001. [Online]. Available: https://agilemanifesto.org/. [Accessed: Oct. 20, 2026].

[4] VMware, Inc. and the Spring community, "Spring Boot Reference Documentation." [Online]. Available: https://docs.spring.io/spring-boot/reference/. [Accessed: Oct. 20, 2026].

[5] Oracle Corporation, "Java Documentation." [Online]. Available: https://docs.oracle.com/en/java/. [Accessed: Oct. 20, 2026].

[6] Oracle Corporation, "MySQL 8.4 Reference Manual." [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/. [Accessed: Oct. 20, 2026].

[7] Object Management Group, "Unified Modeling Language (UML)." [Online]. Available: https://www.omg.org/uml/. [Accessed: Oct. 20, 2026].

8 APPENDICES & SUPPORTING EVIDENCE

This section presents supporting diagrams and project artefacts related to the design and development of the Enterprise Publishing Management System (EPMS). The appendices provide additional evidence of the system's structural design, the existing business environment, and the proposed integrated solution. These artefacts complement the discussions in the main report by illustrating the system's class structure, the existing (As-Is) architecture, and the proposed (To-Be) architecture.

Appendix A: UML Class Diagram

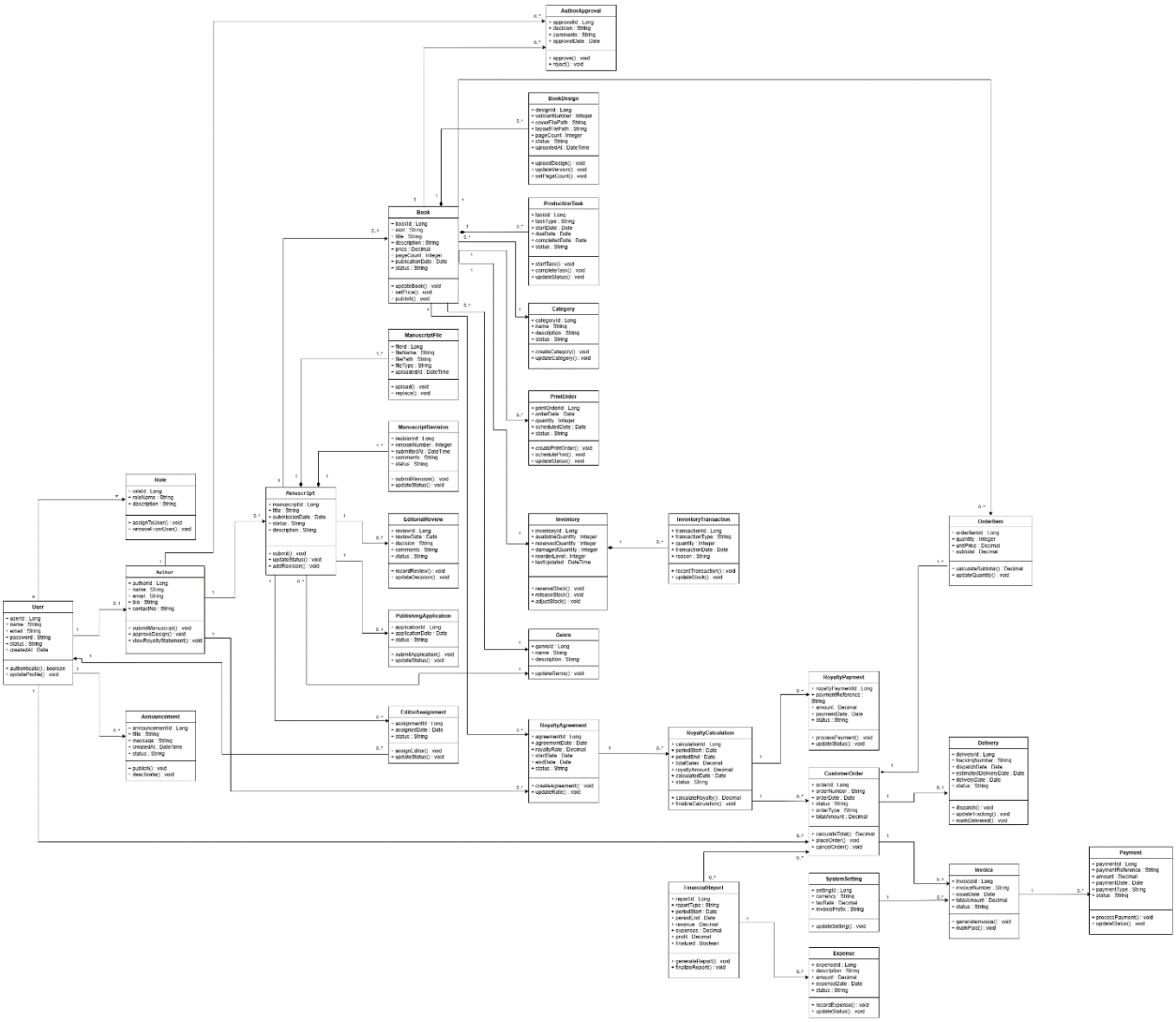


Figure 8.1: UML CLASS DIAGRAM

Diagram link : [UML CLASS DIAGRAM.png]

The UML class diagram illustrates the structural design of the Enterprise Publishing Management System (EPMS), including its classes, attributes, operations, and relationships. It represents the main entities involved in author and manuscript management, editorial workflow and book production, publishing operations, and finance and royalty management. The diagram also shows how related entities interact to support the integrated system.

Appendix B: High Level Architecture Diagram

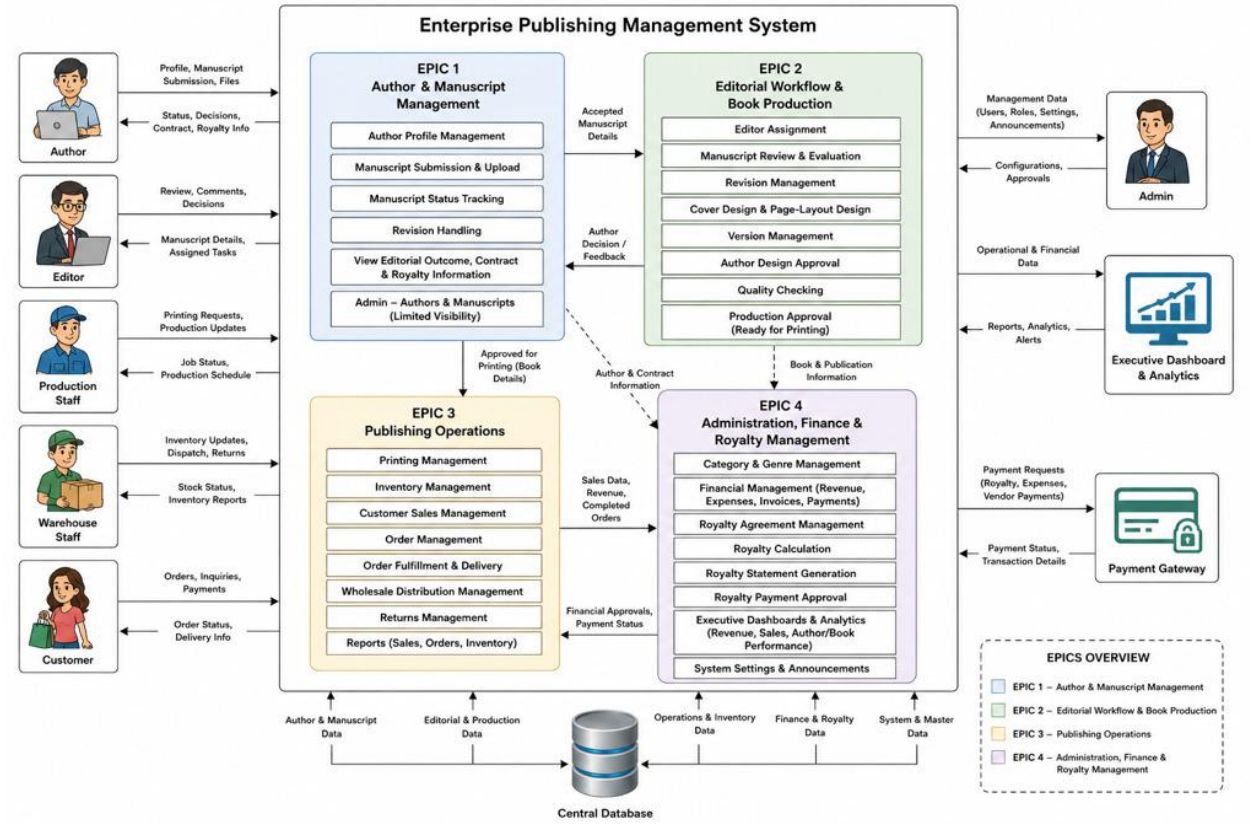


Figure 8.2:High Level Architecture Diagram

The high-level architecture diagram (Figure 8.2:High Level Architecture Diagram) illustrates the overall structure of EPMS and its four functional areas: Author and Manuscript Management, Editorial Workflow and Book Production, Publishing Operations, and Administration, Finance and Royalty Management. It also shows the interactions between the system, its external users, the

central database, and supporting services. The diagram provides an overview of how the functional areas are connected to support the publishing lifecycle.